

ACTIVIDAD EVALUABLE UD3

Funciones

Javier Sánchez

EJERCICIO 1

Analizador de palabras

Crea un módulo `textos.py` con funciones:

- **`contar_palabras(cadena: str) -> int`**, devuelve el número de palabras en la cadena. Las palabras estarán separadas por espacios o algún signo de puntuación (, . ; : ! ; ¿ ?).
- **`caracteres_total(cadena: str) -> int`**, devuelve la suma de todos los caracteres de todas las palabras. Se ignoran signos de puntuación (, . ; : ! ; ¿) y espacios.
- **`palabra_mas_larga(cadena: str) -> str`**, devuelve la palabra más larga. Si hay varias con la misma longitud, devuelve la primera

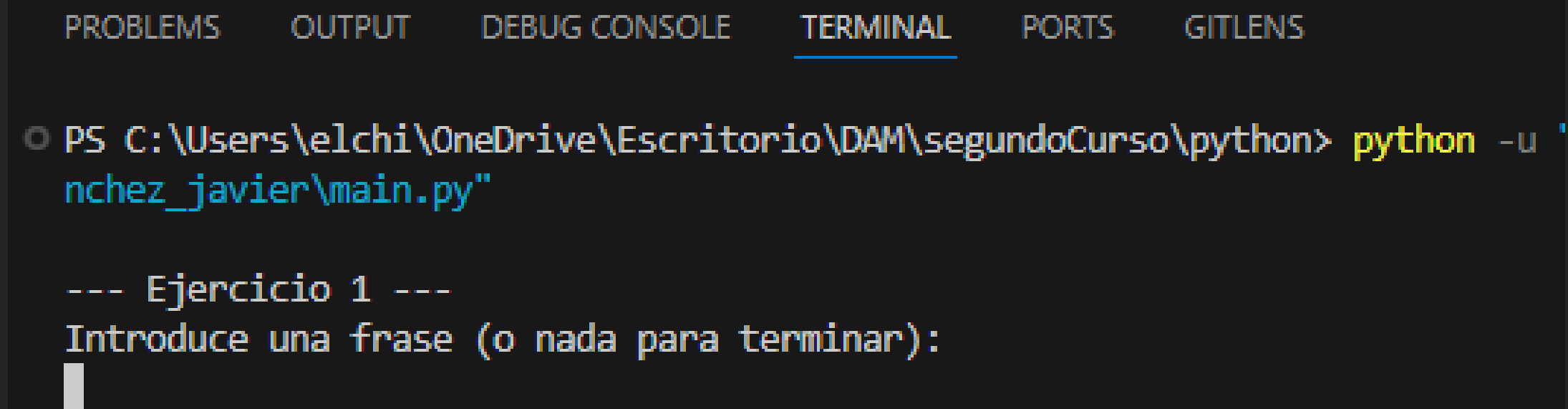
En `main.py`, pide al usuario frases (hasta que ingrese cadena vacía). Por cada frase:

- Mostrar número de palabras de cada frase.
- Mostrar el número total de caracteres
- Mostrar la palabra más larga una cola (Queue)

EJERCICIO 1

Pedimos al usuario que introduzca una frase.

También le damos la opción de terminar no introduciendo nada.



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the command to run a Python script and the script's output. The command is: `PS C:\Users\elchi\OneDrive\Escritorio\DAM\segundoCurso\python> python -u 'nchez_javier\main.py'`. The output is: `--- Ejercicio 1 ---` followed by `Introduce una frase (o nada para terminar):` and a blank line with a cursor.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

○ PS C:\Users\elchi\OneDrive\Escritorio\DAM\segundoCurso\python> python -u '
nchez_javier\main.py'

--- Ejercicio 1 ---
Introduce una frase (o nada para terminar):
█
```

EJERCICIO 1

Introducimos la frase y al dar al ENTER, nos da el resultado y nos vuelve a dar la opción de seguir introduciendo más frases o de terminar.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

○ PS C:\Users\elchi\OneDrive\Escritorio\DAM\segundoCurso\python> python -u
nchez_javier\main.py"

--- Ejercicio 1 ---
Introduce una frase (o nada para terminar):
¡Hola! Vamos a comprobar si este programa funciona correctamente
- Número de palabras: 9
- Número total de caracteres (sin espacios ni signos de puntuación): 54
- Palabra más larga: correctamente

Introduce una frase (o nada para terminar):
█
```

EJERCICIO 1

Hago un segundo intento para comprobar que el ejercicio funciona sólomente con palabras y no permite números.

```
Introduce una frase (o nada para terminar):  
Este es 2 intento del ejercicio  
ERROR: Carácter no válido detectado: '2'. Solo se permiten letras, espacios y signos de puntuación  
  
Introduce una frase (o nada para terminar):  
█
```

He utilizado manejo de excepciones en **utilidades.py** (función **pedir_texto**) para lanzar este mensaje

```
try:  
    # Compruebo letra a letra:  
    for caracter in valor:  
        if not (caracter.isalpha() or caracter.isspace() or caracter in signos_permitidos):  
            raise ValueError(f"Carácter no válido detectado: '{caracter}'. Solo se permiten letras,  
  
    return valor
```

EJERCICIO 1

Si introduzco una **cadena vacía**, pasamos al siguiente ejercicio.

```
Introduce una frase (o nada para terminar):
```

```
Pulsa ENTER para continuar al siguiente ejercicio
```

EJERCICIO 2

Si introduzco una cadena vacía, pasamos al siguiente ejercicio.

Crea una **función recursiva** que reciba una cadena y devuelva la **cadena invertida**.

No se permite usar **slicing** `::-1`

Una vez que la tenemos, la invertimos con la función **invertir_cadena** y utilizamos recursividad:

```
--- Ejercicio 2 ---  
Introduce la cadena de texto que quieres invertir:  
Esta es la demostración del segundo ejercicio  
- Cadena original: Esta es la demostración del segundo ejercicio  
- Cadena invertida: oicicreje odnuges led nóicartsomed al se atsE  
  
Pulsa ENTER para continuar al siguiente ejercicio
```

EJERCICIO 3

Crea una función **ordenar_lista** que reciba una lista de números y devuelva una nueva lista ordenada de menor a mayor.

Puede hacerse iterativa (burbuja, inserción, selección) o recursiva (merge sort o quick sort).

Explica el funcionamiento del método elegido e indicar por que se ha escogido.

- Crea una función recursiva **busqueda_binaria(lista, elemento, inicio, fin)**
- No se permiten bucles (for/while)
- La función debe recibir una lista ordenada y un número a buscar. Debe devolver:
 - La posición del elemento que ocupa en la lista, si se encuentra
 - -1 si no se encuentra

EJERCICIO 3

Método utilizado: ordenación de burbuja iterativo.

He elegido el método iterativo de la burbuja por la facilidad de implementación mediante estructuras de repetición básicas como los **bucles for anidados**. Además, como el ejercicio pide devolver una lista ordenada sin modificar la original, este método permite hacer una copia y trabajar sobre ella modificando las posiciones directamente en memoria sin cambiar la lista original.

¿Cómo funciona?

El algoritmo funciona recorriendo la lista repetidamente. En cada iteración del bucle interno, compara elementos adyacentes. Si el elemento de la izquierda es mayor que el de la derecha, intercambia sus posiciones. Este proceso empuja el valor más grande hacia el final de la lista en cada pasada completa. El bucle externo asegura que este proceso se repita n veces, haciendo que todos los elementos queden ordenados de menor a mayor.

EJERCICIO 3

Pedimos al usuario la lista de números que vamos a ordenar:

```
--- Ejercicio 3 ---  
Introduce los números que quieres ordenar:  
█
```

Antes de nada probamos qué pasaría si **no** introduzco un **número entero**:

```
--- Ejercicio 3 ---  
Introduce los números que quieres ordenar:  
dos, cinco, 10  
ERROR: Asegúrate de introducir solo números enteros separados por comas  
Introduce los números que quieres ordenar:  
█
```

EJERCICIO 3

Ahora, introduzco números enteros al azar.

```
Introduce los números que quieres ordenar:  
24, 12, 36, 80, 28, 1  
  
- Lista introducida: [24, 12, 36, 80, 28, 1]  
  
Pulsa ENTER para ordenar los números introducidos  
␣
```

Pulso ENTER y me los ordena, gracias a la función **ordenar_lista**

```
- Lista ordenada: [1, 12, 24, 28, 36, 80]  
  
Introduce un número para buscar en la lista: █
```

EJERCICIO 3

Por último, introducimos el número a a buscar dentro de la lista:

- **Opción a:** que el número no esté en la lista

```
Introduce un número para buscar en la lista: 2
```

```
El número 2 no está en la lista
```

- **Opción b:** si el número está dentro de la lista, me da la posición que ocupa en la lista ordenada

```
Introduce un número para buscar en la lista: 36
```

```
El número 36 está en la posición 4
```